BLOOM: BMI & Female Health Tracker

Technical Document

A desktop application written in Python with CustomTkinter

COMPE 02: Programming Logic and Design

Members
Justin Blaire M. Ojos
Vena Lee B. Rebaula
Lesther Lazarte
Hayden Tamayo

TABLE OF CONTENTS

1. Introduction
2. Objectives, Scope and Limitations
3. Requirements and System Specifications
4. Program Structure and Code Organization
5. Algorithms and Flowcharts
6. Testing and Test Cases
7. User Guide
8. Health Standards Used
9. References

1. INTRODUCTION

Bloom is a desktop application for women's health. It brings together the things a woman may want to track in one private place: body mass index (BMI) and weight, menstrual periods and ovulation, basal body temperature (BBT), mood and symptoms, vital signs and sleep, physical activity, and nutrition.

Many period and fitness apps keep data online, show advertisements, or give numbers without saying where they come from. Bloom keeps every account's data on the user's own computer, protects each account with a hashed password, and bases every calculation on a published health guideline (CDC, WHO, FIGO, Office on Women's Health, ACC/AHA, FDA). The sources are listed inside the app.

> **Disclaimer.** Bloom gives general wellness information and estimates. It is not a medical device, it does not diagnose, and its fertile-window predictions are not a method of birth control.

2. OBJECTIVES, SCOPE AND LIMITATIONS

2.1 General objective

To design and build a working, well-tested desktop program that helps women track and understand their health using correct, evidence-based calculations.

2.2 Specific objectives

1. To compute BMI and classify it with both the CDC and the Asia-Pacific (WHO Western Pacific Region) cut-offs, together with the healthy weight range.
2. To predict the next period, ovulation and fertile window from the user's own cycle history.
3. To confirm ovulation from basal body temperature using the 3-over-6 rule.
4. To classify blood pressure, heart rate, blood oxygen and sleep, and to track activity and nutrition against WHO, CDC and FDA guidelines.
5. To protect each account with salted, hashed passwords, a lockout after repeated failures, and a one-time recovery code.
6. To validate every input and to prove the logic correct with automated tests, including edge cases.

2.3 Scope

- Several local accounts on one computer; data saved as JSON files in the `data/` folder.
- Screens: Log in, Sign up and password reset, First-run setup, Today, Cycle, Temperature, Journal, Vitals & Sleep, Activity, Nutrition, BMI and Settings.
- Metric and US units, light and dark themes, a "reduce motion" option and CSV export.

2.4 Limitations

- Predictions are statistical estimates; irregular cycles lower their accuracy (the app shows a confidence level).
- BMI does not measure body fat, and the adult categories apply only from age 20 and not during pregnancy.
- The app runs offline on one computer; there is no cloud sync between devices.
- Readings are typed by the user; the app does not connect to devices such as thermometers or watches.

3. REQUIREMENTS AND SYSTEM SPECIFICATIONS

3.1 Hardware and software

Item	Requirement
Operating system	Windows 10 or 11 (also runs on macOS with Python 3.13)
Python	3.14 (with Tk 8.6)
Screen	At least 860 × 560 pixels (the window's minimum size)
Libraries	**customtkinter 6.0.0** (widgets), **pillow 13.0.0** (icons, charts, 3D images), **numpy 2.5.4** (3D ray marcher, gradients)
Development tools	**pytest 9.2.0** (tests), **ruff 0.17.3** (formatting and style checks)

3.2 Installing and running

```
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
python main.py                    start the app
python -m tracker.ui.preview      open a demo account (demo / bloom2026)
pytest                            run the 161 tests
```

3.3 Functional requirements

No.	Requirement	Where it is met
FR1	Create an account, log in, log out, reset a forgotten password	auth.py, auth_pages.py
FR2	Lock an account for 30 s after 5 wrong attempts	auth.check_login
FR3	Compute BMI, its category and healthy weight range	bmi.py, bmi_page.py
FR4	Log periods; predict next period, ovulation and fertile window	cycle.py, cycle_page.py
FR5	Show a coloured cycle calendar	calendar_logic.py
FR6	Log BBT and confirm ovulation	bbt.py, bbt_page.py
FR7	Log mood, energy, stress and symptoms	journal.py, journal_page.py
FR8	Classify blood pressure, heart rate, SpO2 and sleep	vitals.py, vitals_page.py
FR9	Track workouts, steps and weekly active minutes	activity.py, activity_page.py
FR10	Track water, meals, caffeine, alcohol, fruit and vegetables	nutrition.py, profile.py
FR11	Give personal insights and a daily checklist	insights.py, home_page.py
FR12	Validate every input with a clear message	validators.py
FR13	Save data safely and survive a damaged file	storage.py

3.4 Non-functional requirements

- **Security:** passwords and recovery codes are stored only as PBKDF2-SHA256 hashes (600,000 iterations, 16-byte random salt), as advised by OWASP.
- **Reliability:** files are written to a temporary file and then swapped in, so a crash cannot leave a half-written file.
- **Usability:** errors appear next to the field that caused them; logging takes one or two clicks; text contrast is at least 4.5:1.
- **Maintainability:** the logic is separate from the screens, and every public function has a docstring.

4. PROGRAM STRUCTURE AND CODE ORGANIZATION

The program is split into two layers. The **logic layer** (`tracker/`) contains all calculations and rules in plain Python. The **user interface layer** (`tracker/ui/`) builds the windows and calls the logic. This separation means every calculation can be tested and flowcharted on its own, without opening a window.

4.1 Size of the program

Part	Files	Lines of code*	Total lines
Logic (`tracker/`)	17	1,519	2,440
User interface (`tracker/ui/`)	21	6,621	7,808
`main.py`	1	5	12
Tests (`tests/`)	8	626	881

** Lines of code exclude blank lines, comments and docstrings.*

4.2 Logic modules (tracker/)

Module	Purpose	Main functions
auth.py	Accounts and security	register_user, check_login, reset_password_with_code, change_password, delete_account
storage.py	The only module that reads and writes files	load_users, save_users, load_user_data, save_user_data, export_csv
validators.py	Turns typed text into checked values	parse_number, parse_weight, parse_height, parse_temperature, parse_date, parse_birth_date
units.py	kg/lb, cm/ft-in and °C/°F conversion	kg_to_lb, ft_in_to_cm, c_to_f, format_weight
profile.py	Age, calorie target, water goal, goals	bmr_mifflin_st_jeor, daily_calorie_target, suggested_water_ml
bmi.py	BMI and categories	calculate_bmi, classify_bmi, healthy_weight_range, calculate_age
cycle.py	Periods, cycle statistics and predictions	add_period, cycle_lengths, predict_cycle, phase_on, cycle_day
calendar_logic.py	Month grid and day colours	month_grid, day_status
bbt.py	Basal body temperature	readings_between, detect_thermal_shift, is_fever
daylog.py, journal.py	Daily logs, mood and symptom patterns	peek_day, sleep_mood_pairs

vitals.py	Blood pressure, heart rate, SpO2, sleep	bp_category, rhr_note, spo2_note, sleep_debt
activity.py	Workouts and active minutes	workout_kcal, intensity, moderate_minutes
nutrition.py	Water, caffeine, alcohol, fruit and vegetables	expected_water_by_now, caffeine_note
insights.py	Smart tips and the daily checklist	build_insights, daily_checklist
demo.py	90 days of made-up demo data	make_demo_data

4.3 User interface modules (tracker/ui/)

Module	Purpose
app.py	The main window (class **App**, a subclass of `ctk.CTk`), the sidebar, page switching, saving and keyboard shortcuts
page.py	Base class **Page**; every screen inherits from it and overrides `build()`
auth_pages.py, onboarding.py	Log in, sign up, password reset, recovery code, and the 7-step setup
home_page.py … settings_page.py	One file per sidebar page (9 pages)
widgets.py	Reusable pieces: cards, buttons, inputs, charts, rings, dialogs, toasts
theme.py, anim.py	Colours for light and dark mode, fonts, spacing; animation easing and timers
icons.py, graphics.py, render3d.py	Icons, charts and 3D shapes drawn with Pillow and numpy
preview.py	Opens the app with a demo account in a temporary folder

4.4 Design rules

1. **Logic never imports the user interface.** Modules in `tracker/` do not import customtkinter, tkinter or Pillow.
2. **Every public function has a docstring** that says what it returns and which errors it raises.
3. **Only storage.py touches files.** Inside the program, dates are `datetime.date` objects; in the files they are ISO text such as `"2026-10-31"`.

Rules 1 and 2 are checked automatically by `tests/test_architecture.py`.

4.5 Programming constructs used

- **Selection:** if / elif / else chains (e.g. `bp_category` checks from the most serious category down).

- **Repetition:** `for` loops over tables and lists (BMI categories, cycle lengths, the 3-over-6 window, the last 7 days); nested loops build the calendar grid.
- **Functions** with type hints, default arguments and return values; **constants** in capitals instead of "magic numbers".
- **Data structures:** lists, dictionaries, tuples and frozen `dataclass` objects (`CyclePrediction`, `ThermalShift`).
- **Exception handling:** the logic raises `ValueError` or `AuthError`; the screens catch them with try/except and show the message.
- **Classes and inheritance:** `App` inherits from `ctk.CTk`; every page inherits from `Page`.

4.6 Data storage

`data/users.json` holds the accounts (username, password hash, salt, iterations, recovery-code hash, failed attempts and lock time). Each user also has a private data file named after the user id (e.g. `u_3f9a1c2b`) holding the profile, goals, settings, periods, weights and daily logs. Example of a period and a day log:

```
"periods": [{"start": "2026-10-20", "end": "2026-10-24"}],
"days": {"2026-11-03": {"bbt_c": 36.62, "mood": 4, "sleep_h": 7.5,
                        "water_ml": 750, "bp_sys": 112, "bp_dia": 74}}
```

5. ALGORITHMS AND FLOWCHARTS

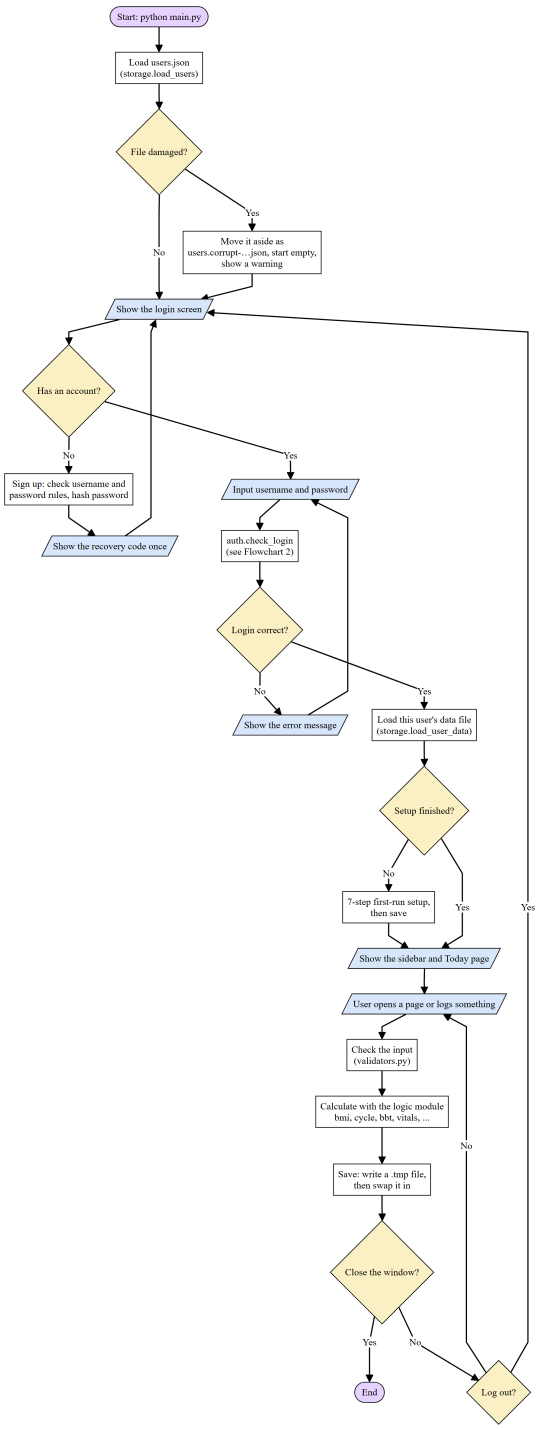
Each flowchart follows its function's code step by step, so it can be compared with the code directly. Symbols: **oval** = start/end, **parallelogram** = input/output, **rectangle** = process, **diamond** = decision.

5.1 The whole program

The program starts in **main.py**, which creates the **App** window. App.start() loads the accounts; a damaged file is moved aside, not deleted. The user signs up or logs in, finishes the setup once, and then repeats the main loop: open a page or log something, check the input, calculate, and save. Logging out returns to the login screen; closing the window ends the program.

Flowchart 1. The Whole Program

Code: tracker/ui/app.py (App.start, log_in, enter_main, show_page)

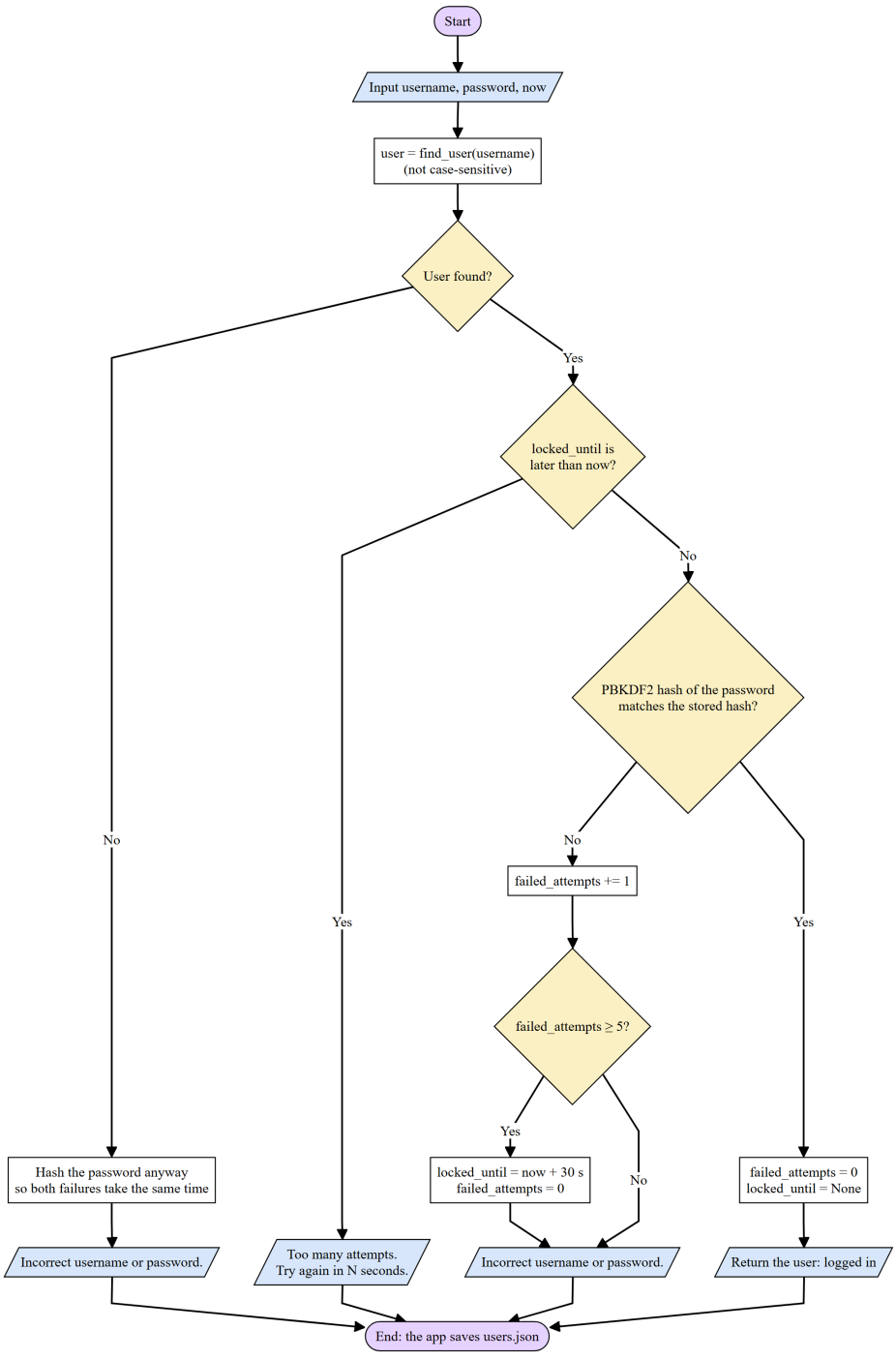


5.2 Logging in with the 5-try lockout

check_login() hashes the password even when the username does not exist, so an attacker cannot tell from the timing which part was wrong. Both failures show the same message. Every wrong password adds 1 to failed_attempts; the 5th locks the account for 30 seconds and resets the counter.

Flowchart 2. Logging In with the 5-Try Lockout

Code: auth.check_login

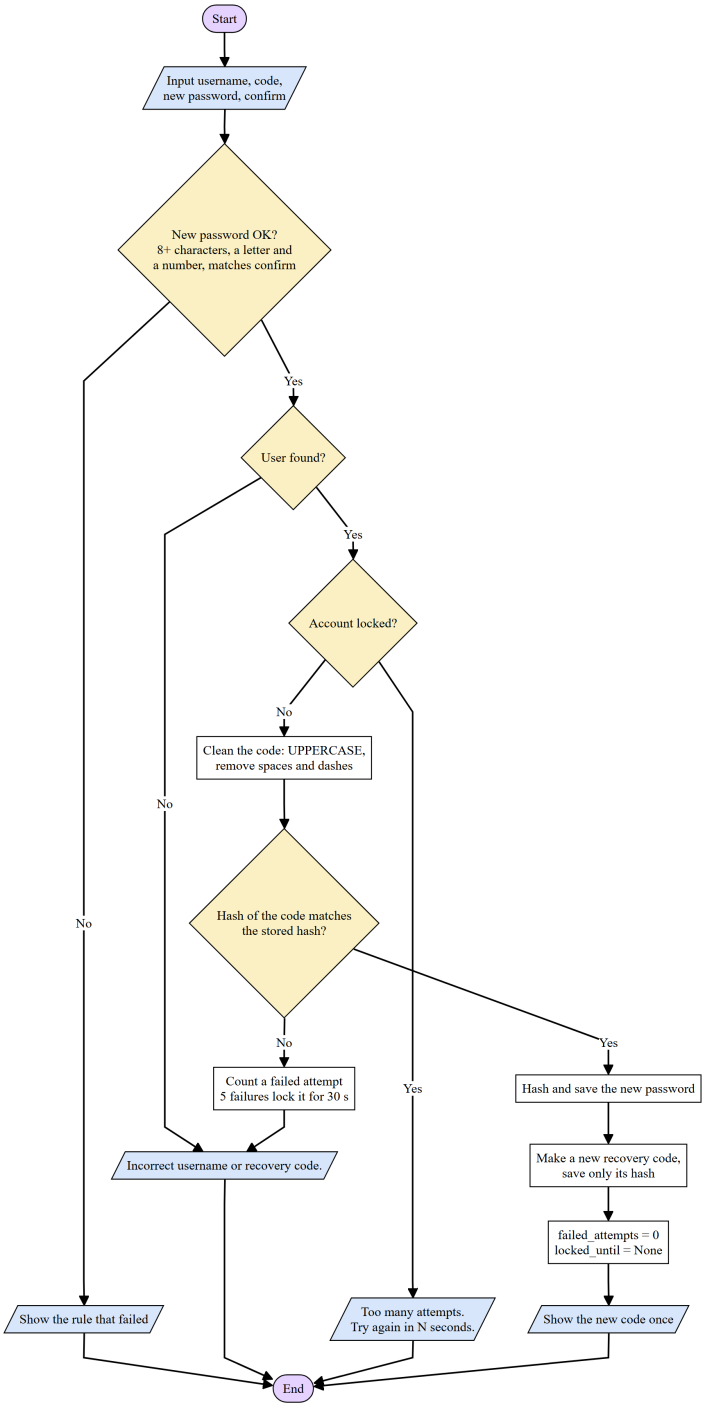


5.3 Resetting a password with the recovery code

The new password's rules are checked *first*, so a typing mistake there does not count as a failed attempt. The code is cleaned (uppercase, no spaces or dashes) before hashing. After a successful reset a **new** code replaces the old one, so each code works only once.

Flowchart 3. Resetting a Password with the Recovery Code

Code: auth.reset_password_with_code

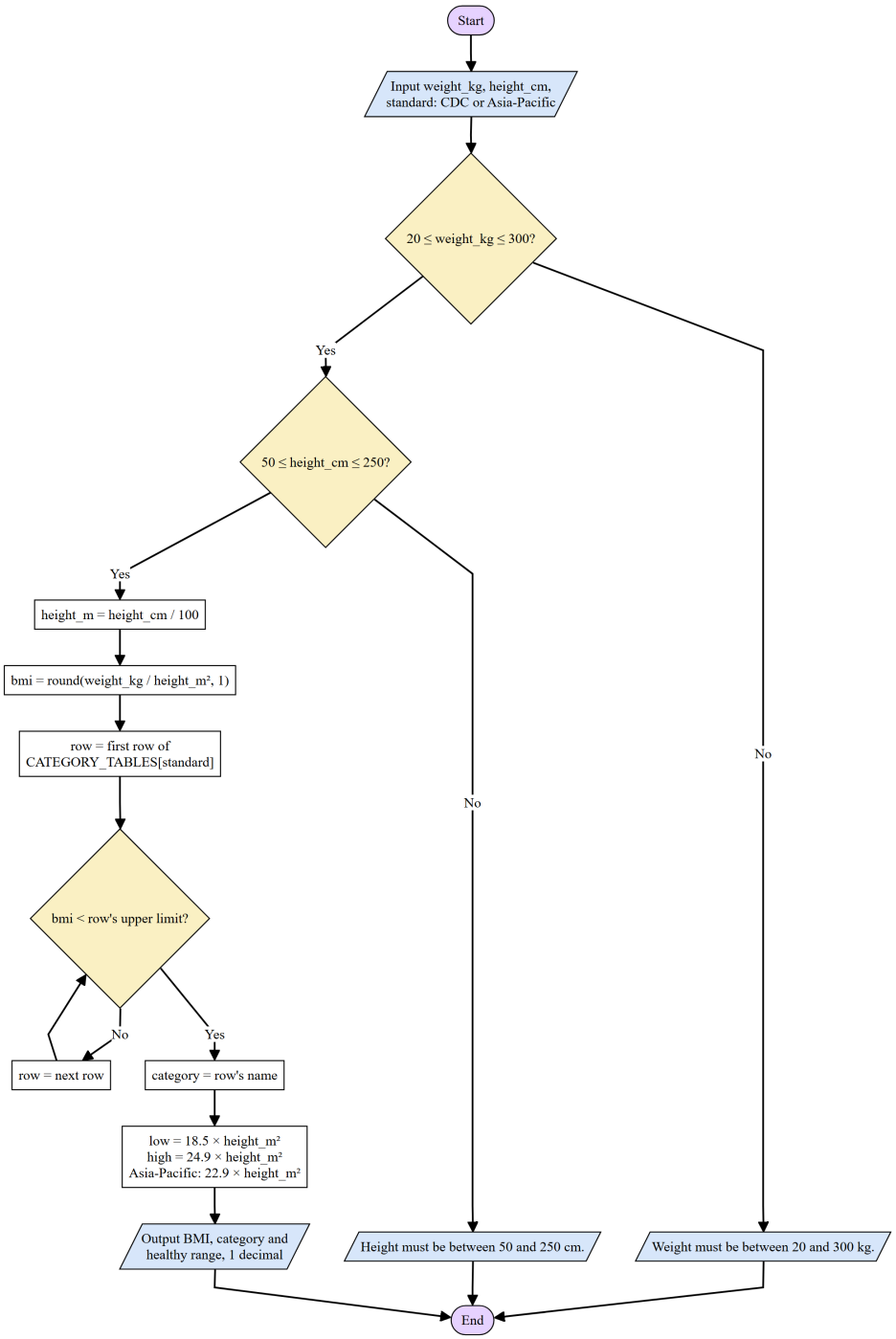


5.4 BMI, its category and the healthy range

The inputs are range-checked, the BMI is rounded to 1 decimal, and the category comes from a **loop over a table** instead of a chain of if-statements. The rounded value is classified so that the label always matches the number on screen (TC02–TC03).

Flowchart 4. BMI, Its Category and the Healthy Range

Code: bmi.calculate_bmi, classify_bmi, healthy_weight_range

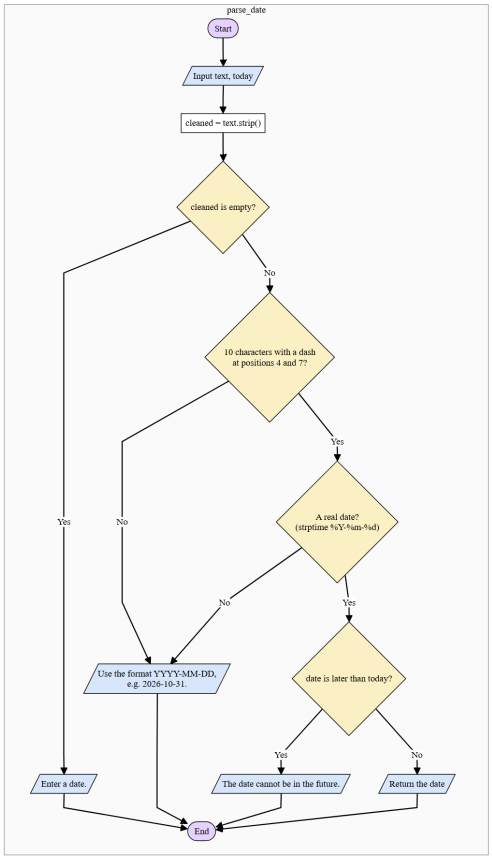


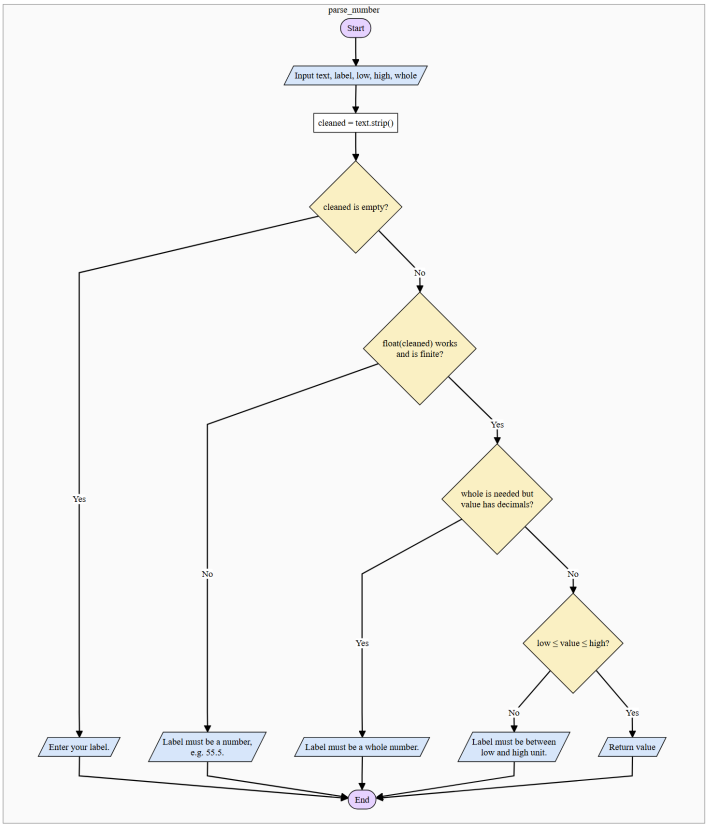
5.5 Checking typed numbers and dates

Every value typed by the user passes through validators.py. Each check that fails raises a ValueError with a message that says what is wrong and how to fix it; the screen shows that message next to the field.

Flowchart 5. Checking Typed Numbers and Dates

Code: validators.parse_number, parse_date



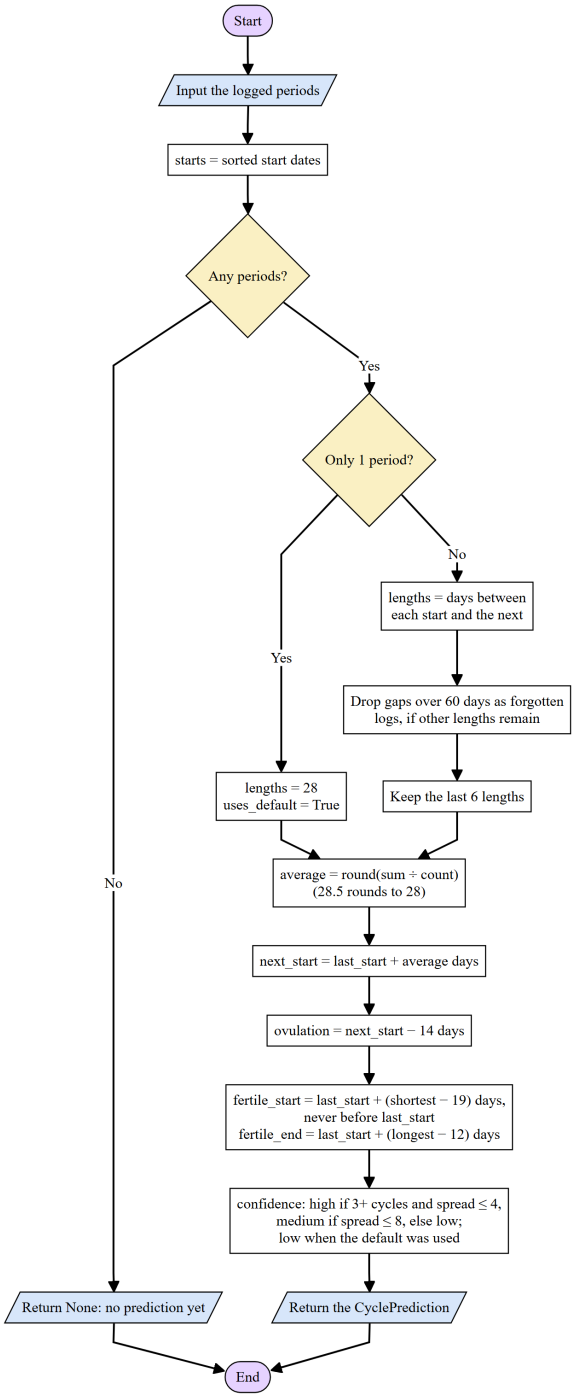


5.6 Predicting the next period and fertile window

The prediction averages the last 6 cycle lengths, ignoring gaps over 60 days (a forgotten log). Ovulation is counted **back 14 days** from the next period because the luteal phase is about 14 days long. The fertile window uses the calendar method: shortest cycle − 18 to longest cycle − 11 (the extra − 1 is because day 1 is the start date itself).

Flowchart 6. Predicting the Next Period and Fertile Window

Code: cycle.predict_cycle

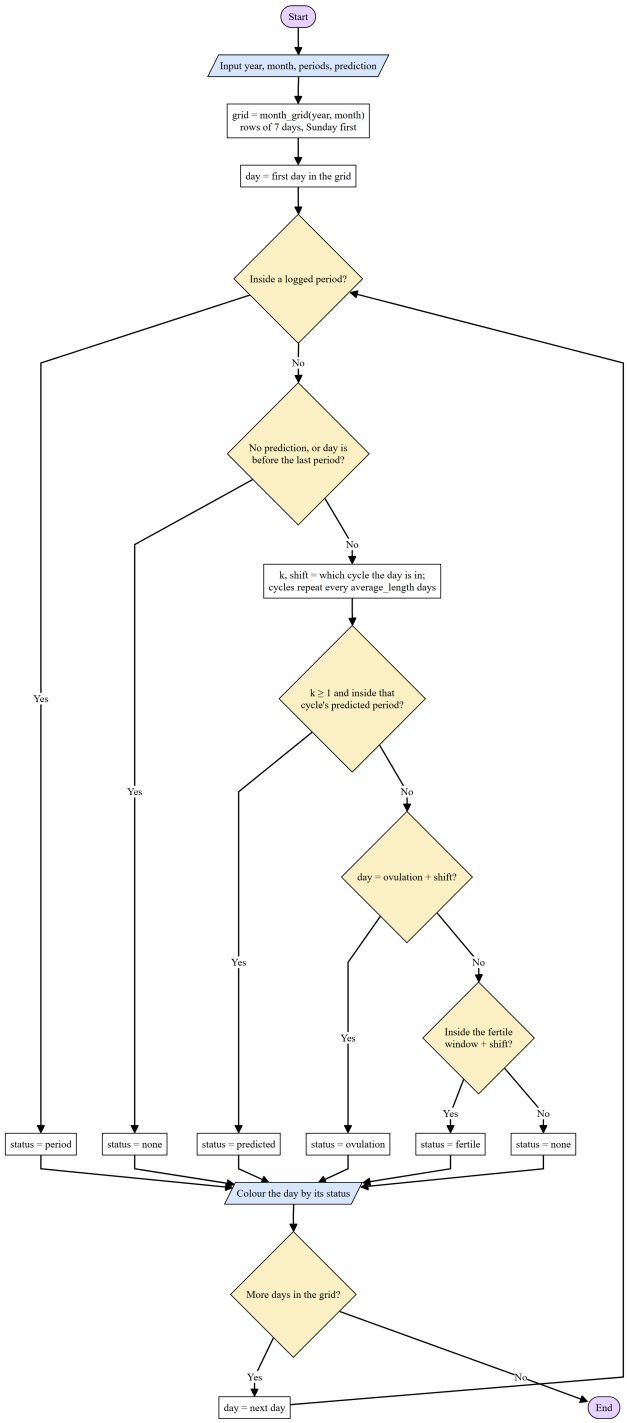


5.7 The month grid and each day's colour

The grid comes from Python's calendar module. Each day is tested in a fixed order: logged period, predicted period, ovulation, fertile window. The first match decides the colour. Predictions repeat every average cycle length, so future months are coloured too.

Flowchart 7. The Month Grid and Each Day's Colour

Code: calendar_logic.month_grid, day_status

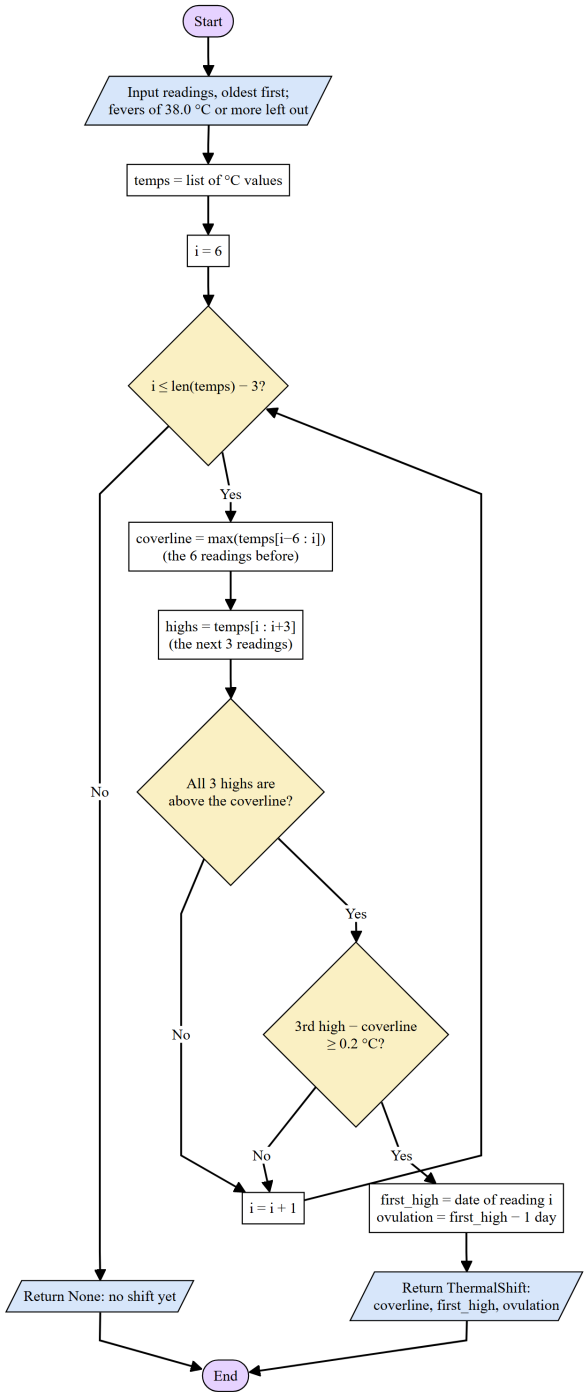


5.8 Confirming ovulation with the 3-over-6 rule

A loop slides along the readings. The coverline is the highest of the 6 readings before position i; the next 3 readings must all be above it, and the 3rd must be at least 0.2 °C higher. Fever readings (38.0 °C or more) are removed first because illness, not progesterone, raised them.

Flowchart 8. Confirming Ovulation with the 3-over-6 Rule

Code: bbt.detect_thermal_shift

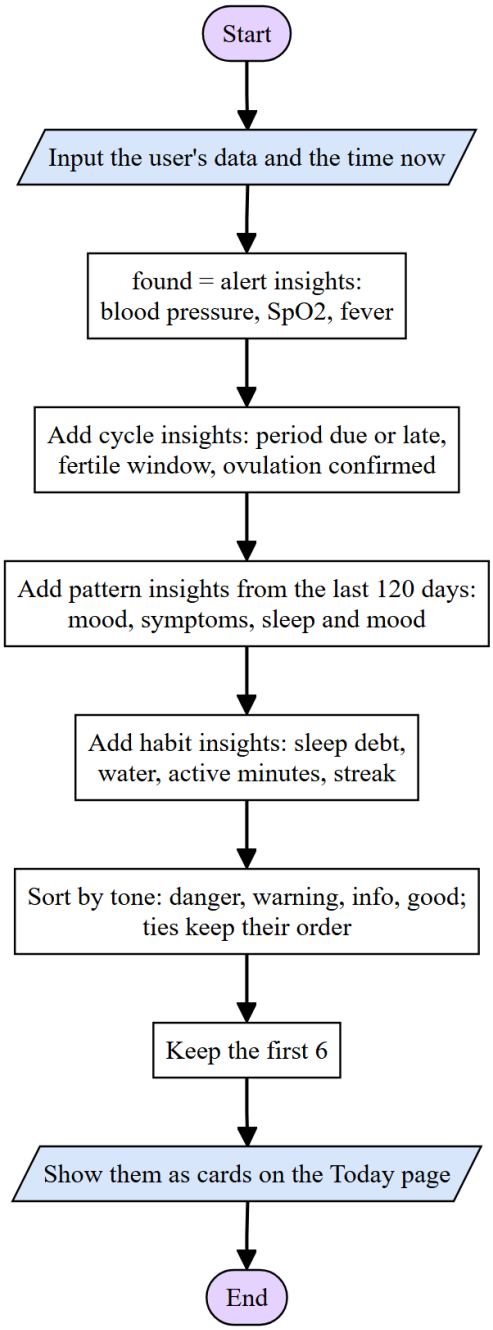


5.9 Choosing the insights for the Today page

Four groups of rules each add their insights to one list. The list is sorted by tone so urgent alerts come first (Python's sort is stable, so ties keep their order), and only the first 6 are shown.

Flowchart 9. Choosing the Insights for the Today Page

Code: insights.build_insights



6. TESTING AND TEST CASES

6.1 Automated unit tests

The logic is tested with **pytest**. There are **161 tests** in 8 files (test_architecture, test_auth, test_bmi, test_cycle, test_health, test_insights, test_storage, test_validators). The storage tests use a temporary folder, so real data is never touched. The latest run:

```
$ pytest
161 passed in 3.36s
```

6.2 Test case table

The table below lists 48 representative test cases, including edge cases (marked *(edge)*). The **actual output** column was produced by running each input through the real functions of the program, not typed by hand. **48 of 48 passed.** Dates assume today is 2026-11-03.

No.	Module	Test case	Input	Expected output	Actual output	Result
TC01	BMI	Normal BMI	55 kg, 160 cm	21.5	21.5	**Pass**
TC02	BMI	rounds up to the limit *(edge)*	47.36 kg, 160 cm (raw 18.4999)	18.5	18.5	**Pass**
TC03	BMI	label matches rounded BMI *(edge)*	BMI 18.5, CDC	Healthy weight	Healthy weight	**Pass**
TC04	BMI	CDC overweight	BMI 27.0, CDC	Overweight	Overweight	**Pass**
TC05	BMI	Asia-Pacific lower cut-off	BMI 23.0, Asia-Pacific	Overweight	Overweight	**Pass**
TC06	BMI	Asia-Pacific obesity class I	BMI 26.0, Asia-Pacific	Obesity class I	Obesity class I	**Pass**
TC07	BMI	Healthy weight range	160 cm, CDC	(47.4, 63.7)	(47.4, 63.7)	**Pass**
TC08	BMI	Invalid: weight too high	350 kg, 160 cm	Error: Weight must be between 20 and 300 kg.	Error: Weight must be between 20 and 300 kg.	**Pass**
TC09	BMI	birthday not yet reached *(edge)*	born 2004-12-01, today 2026-11-03	21	21	**Pass**
TC10	BMI	adult categories at 19 *(edge)*	age 19	False	False	**Pass**
TC11	Input	Text instead of a number	weight "abc"	Error: Weight must be a number, e.g. 55.5.	Error: Weight must be a number, e.g. 55.5.	**Pass**

No.	Module	Test case	Input	Expected output	Actual output	Result
TC12	Input	Blank field	weight ""	Error: Enter your weight.	Error: Enter your weight.	**Pass**
TC13	Input	Height out of range	height "300"	Error: Height must be between 50 and 250 cm.	Error: Height must be between 50 and 250 cm.	**Pass**
TC14	Input	height typed in metres *(edge)*	height "1.6"	Error: Height must be between 50 and 250 cm. Did you enter metres? Type 160 for 1.60 m.	Error: Height must be between 50 and 250 cm. Did you enter metres? Type 160 for 1.60 m.	**Pass**
TC15	Input	Pounds converted to kg	"121" lb	54.9	54.9	**Pass**
TC16	Input	impossible date *(edge)*	"2026-02-30"	Error: Use the format YYYY-MM-DD, e.g. 2026-10-31.	Error: Use the format YYYY-MM-DD, e.g. 2026-10-31.	**Pass**
TC17	Input	Future date	"2026-12-25"	Error: The date cannot be in the future.	Error: The date cannot be in the future.	**Pass**
TC18	Input	"nan" is not a number *(edge)*	weight "nan"	Error: Weight must be a number, e.g. 55.5.	Error: Weight must be a number, e.g. 55.5.	**Pass**
TC19	Cycle	Cycle lengths	starts Jan 3, Jan 28, Mar 4, Apr 3	[25, 35, 30]	[25, 35, 30]	**Pass**
TC20	Cycle	Next period	starts Jan 1, Jan 29, Feb 26 (28-day cycles)	2026-03-26	2026-03-26	**Pass**
TC21	Cycle	Ovulation = next period - 14	same	2026-03-12	2026-03-12	**Pass**
TC22	Cycle	Fertile window (calendar method)	same	2026-03-07 to 2026-03-14	2026-03-07 to 2026-03-14	**Pass**
TC23	Cycle	only one period logged *(edge)*	start Oct 1	28 days, default used	28 days, default used	**Pass**
TC24	Cycle	no periods *(edge)*	no periods	None	None	**Pass**
TC25	Cycle	x.5 average rounds to even *(edge)*	cycles 28 and 29 (average 28.5)	28	28	**Pass**
TC26	Cycle	Confidence with steady cycles	4 cycles of 28 days	high	high	**Pass**
TC27	Cycle	Lengths outside 24-38 days	[22, 28, 40]	[22, 40]	[22, 40]	**Pass**

No.	Module	Test case	Input	Expected output	Actual output	Result
TC28	Cycle	Duplicate period	Oct 1 logged twice	Error: This period is already logged.	Error: This period is already logged.	**Pass**
TC29	Cycle	period over 15 days *(edge)*	Oct 1 to Oct 20	Error: A period cannot last more than 15 days.	Error: A period cannot last more than 15 days.	**Pass**
TC30	Cycle	Cycle day	period Oct 20, day Nov 3	15	15	**Pass**
TC31	BBT	3-over-6 shift found	6 lows (max 36.5), then 36.7, 36.8, 36.8	coverline 36.5, ovulation 2026-10-06	coverline 36.5, ovulation 2026-10-06	**Pass**
TC32	BBT	rise under 0.2 C *(edge)*	6 lows (max 36.5), then 36.6 x3	None	None	**Pass**
TC33	BBT	fever left out *(edge)*	38.0 C	True	True	**Pass**
TC34	Vitals	Normal blood pressure	115/75	('Normal', 'good')	('Normal', 'good')	**Pass**
TC35	Vitals	diastolic alone raises it *(edge)*	118/92	('Stage 2 hypertension', 'warning')	('Stage 2 hypertension', 'warning')	**Pass**
TC36	Vitals	Severe	185/100	('Severe hypertension', 'danger')	('Severe hypertension', 'danger')	**Pass**
TC37	Vitals	Sleep debt (extra sleep doesn't cancel)	goal 8 h; nights 6, 9, 7	3.0	3.0	**Pass**
TC38	Activity	MET calories	Brisk walking 4.3 MET, 55 kg, 30 min	124	124	**Pass**
TC39	Activity	Vigorous minutes count double	20 min running (9.8 MET) + 30 min walking (3.5)	70	70	**Pass**
TC40	Nutrition	Calorie target (Mifflin-St Jeor)	55 kg, 160 cm, 22 y, lightly active	1760	1760	**Pass**
TC41	Nutrition	Water goal	55 kg	1750	1750	**Pass**
TC42	Account	Username too short	"ab"	Error: Username must be 3 to 20 characters.	Error: Username must be 3 to 20 characters.	**Pass**
TC43	Account	Password without a number	"password"	Error: Password needs at least one letter and one number.	Error: Password needs at least one letter and one number.	**Pass**
TC44	Account	Correct login	maria / bloom2026	maria	maria	**Pass**
TC45	Account	Wrong password	maria / wrong123	Error: Incorrect username or password.	Error: Incorrect username or password.	**Pass**

No.	Module	Test case	Input	Expected output	Actual output	Result
TC46	Account	locked after 5 wrong tries *(edge)*	6th try, right password	Error: Too many attempts. Try again in 30 seconds.	Error: Too many attempts. Try again in 30 seconds.	**Pass**
TC47	Account	Unlocked after 30 seconds	same, 31 s later	maria	maria	**Pass**
TC48	Account	Password stored as a hash	stored record	True	True	**Pass**

6.3 System testing

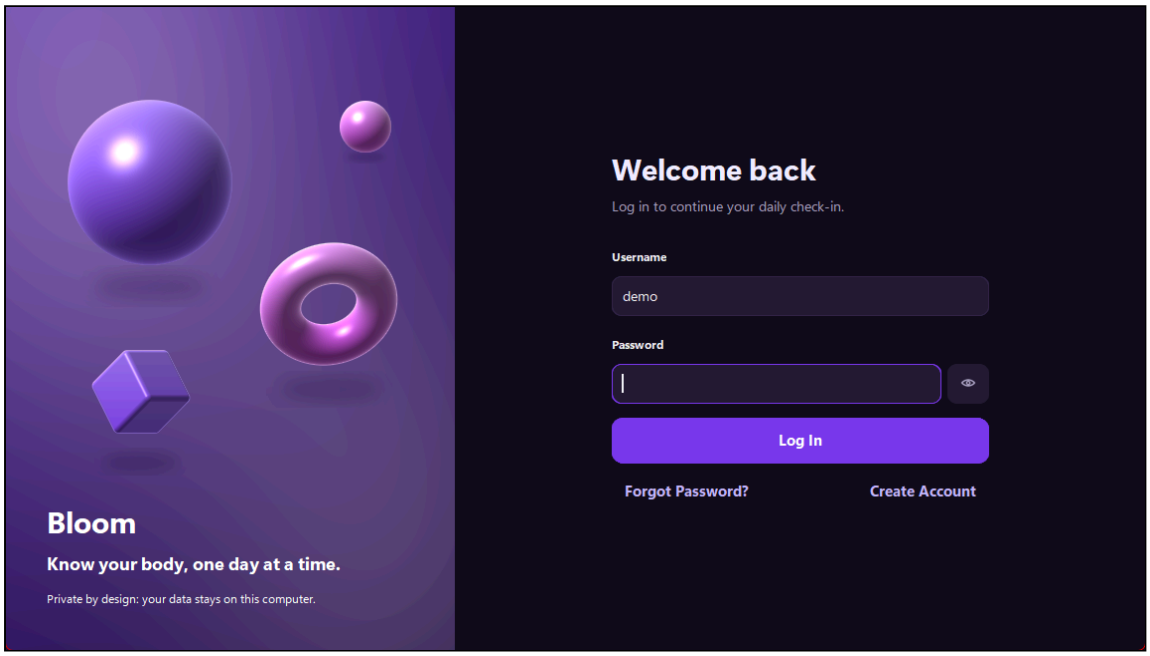
The whole application was also driven from start to finish: logging in with the demo account, opening every page, logging data on each page, switching to dark mode, shrinking the window to the compact layout, logging out, then creating a new account, saving the recovery code and finishing the full setup. No errors occurred.

7. USER GUIDE

Start the app with `python main.py`. The screenshots below use the demo account (`python -m tracker.ui.preview`, username **demo**, password **bloom2026**) in dark mode. Keyboard shortcuts: **Ctrl+1** to **Ctrl+9** open the sidebar pages, **Enter** submits a form and **Esc** closes a dialog.

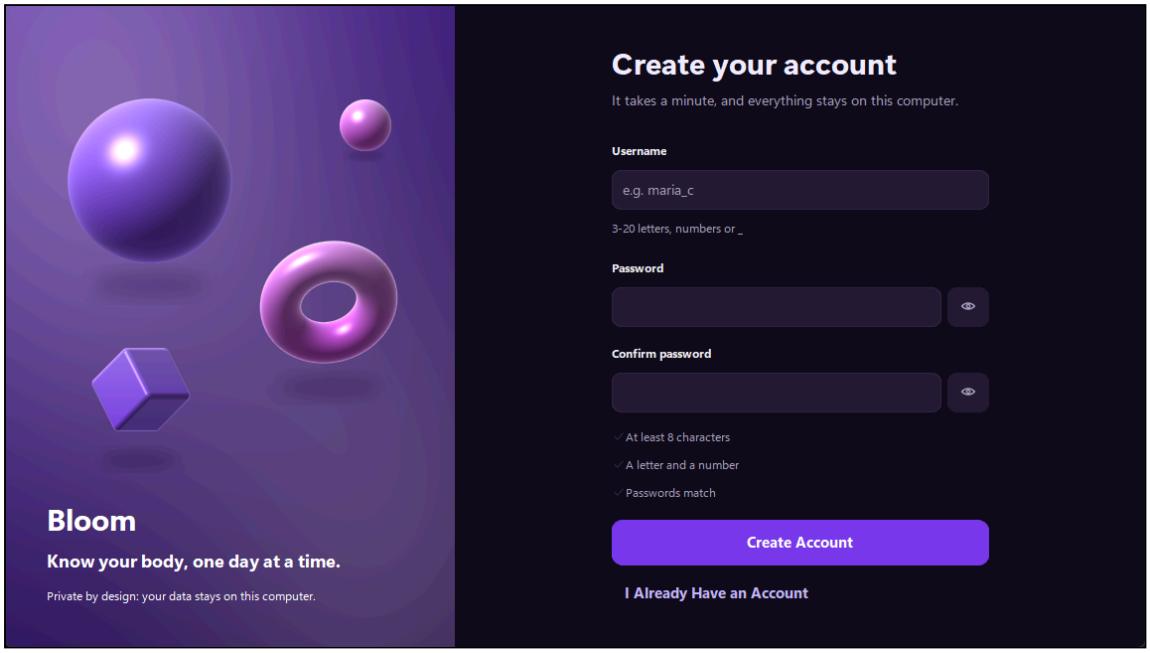
7.1 Log in

Type your username and password and press **Log In** (or Enter). New users click **Create Account**. If you forget your password, click **Forgot Password?** and use your recovery code.



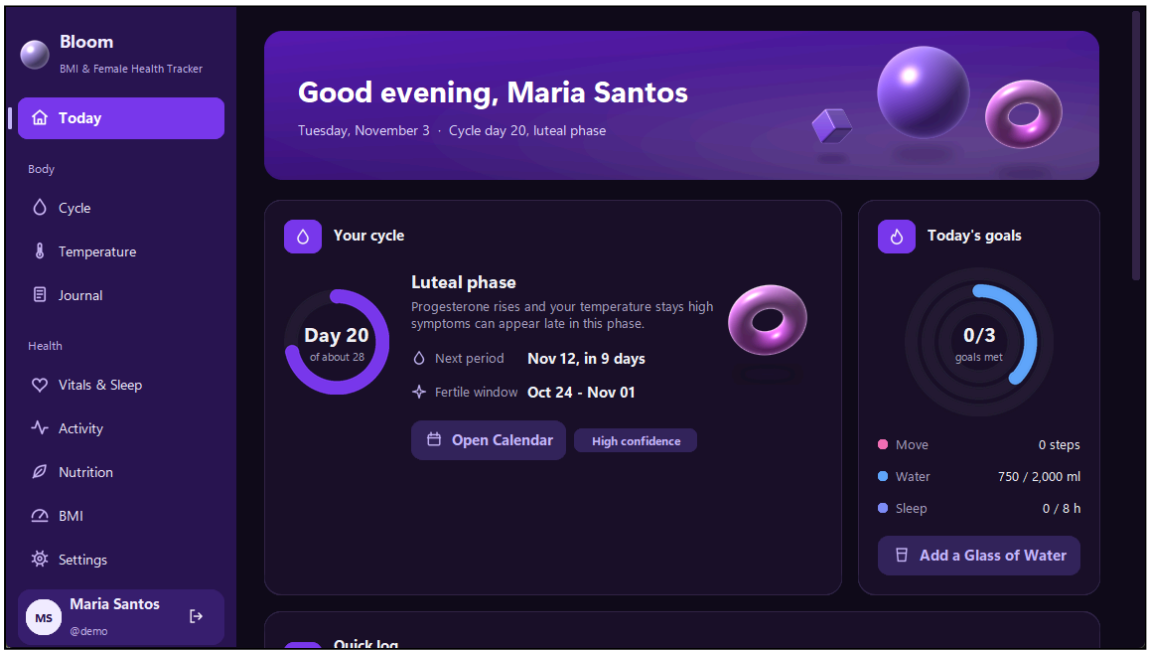
7.2 Create an account

Choose a username (3–20 letters, numbers or _) and a password (8+ characters with a letter and a number). Write down the recovery code shown next; it appears only once. A 7-step setup then asks for your name, birth date, height and weight, cycle, activity level and goals.



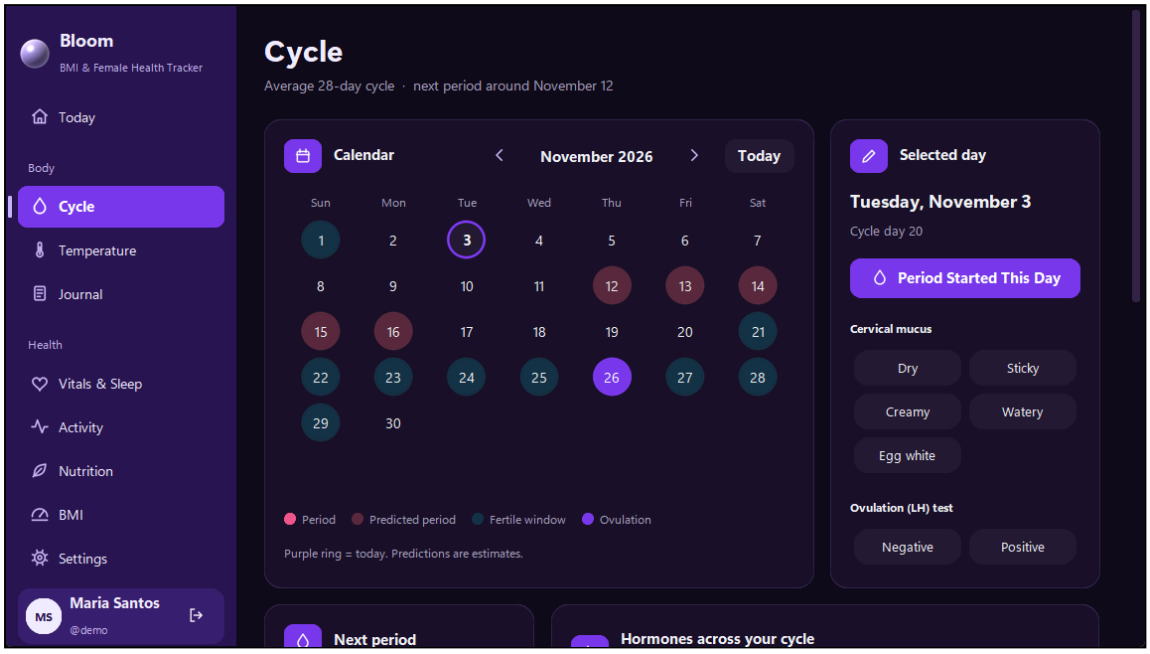
7.3 Today

Shows your greeting and cycle day, the cycle ring and phase, today's goals, quick logging buttons, smart insights, a daily checklist and your last 7 days.



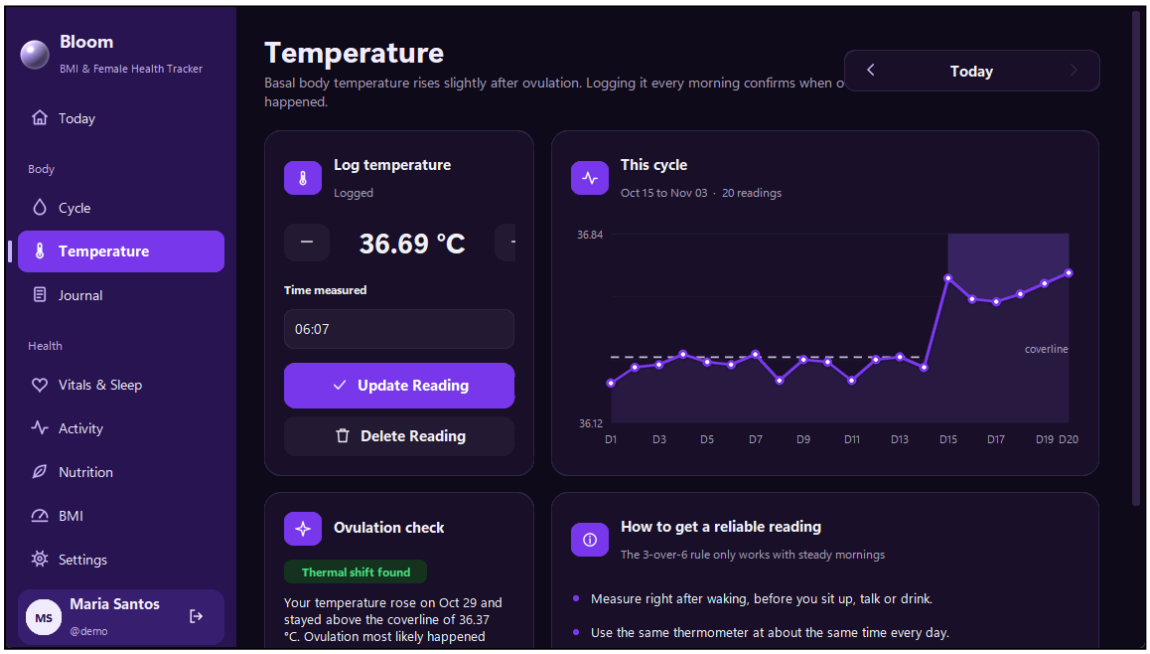
7.4 Cycle

Log a period's start and end, flow, cervical mucus and LH tests. The calendar is coloured: logged period, predicted period, fertile window and ovulation. Predictions show a confidence level.



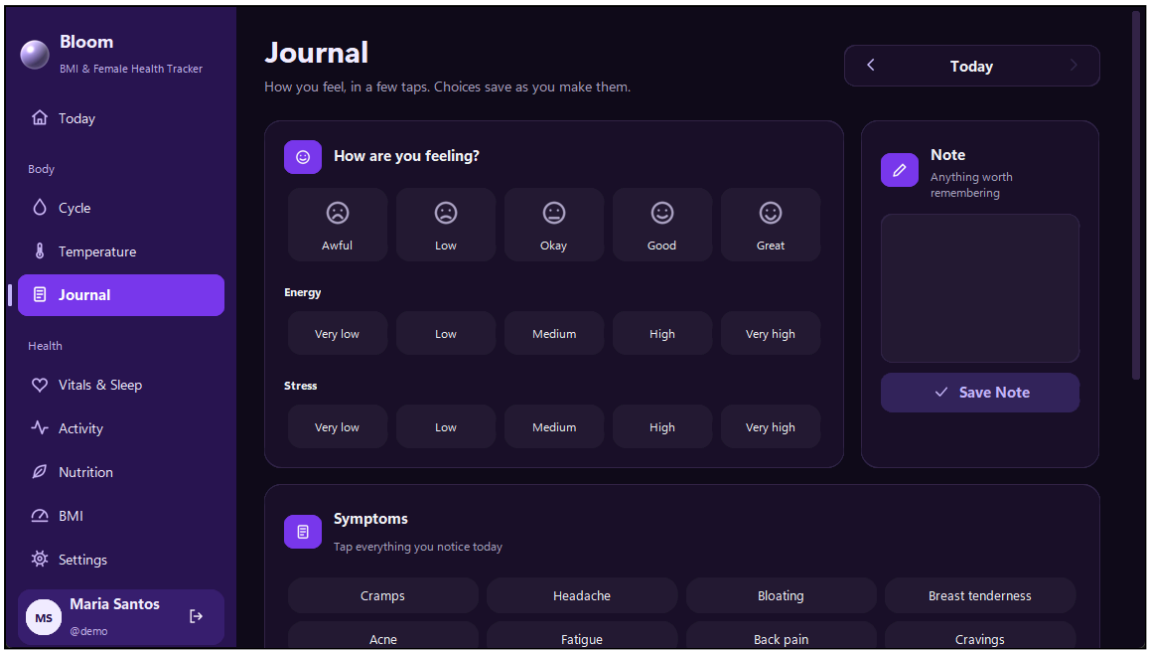
7.5 Temperature (BBT)

Log your temperature each morning before getting up. The chart draws the coverline and marks ovulation once the 3-over-6 rule confirms it.



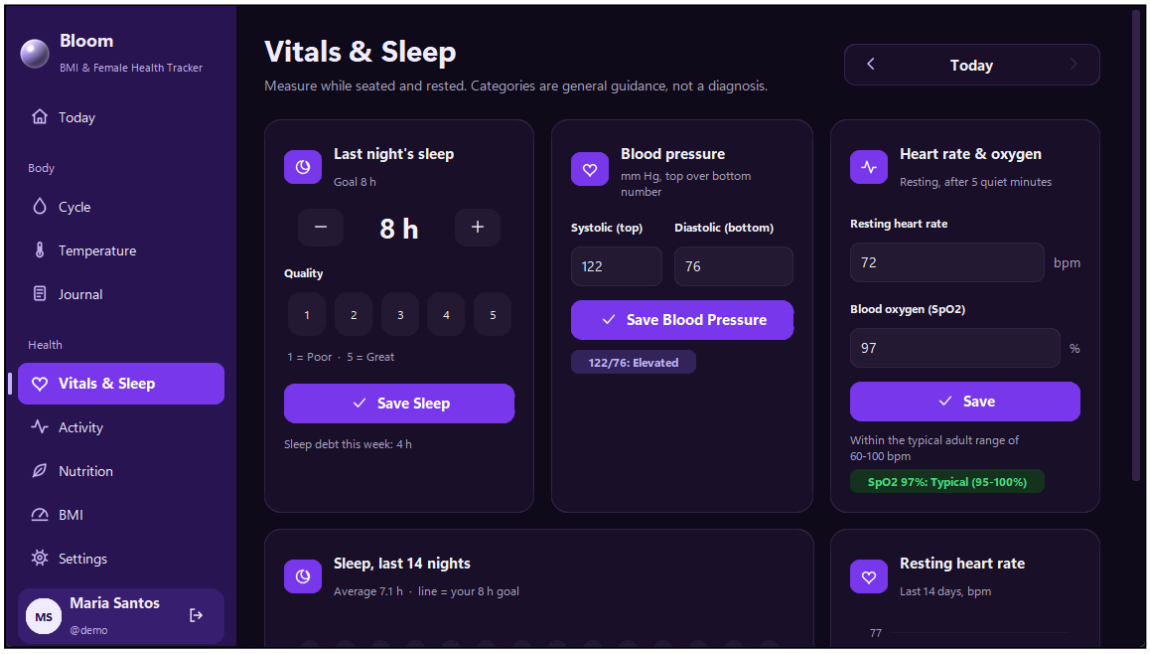
7.6 Journal

Log mood, energy, stress, symptoms and a note for any day. The page shows which symptoms cluster in which phase and a 14-day mood trend.



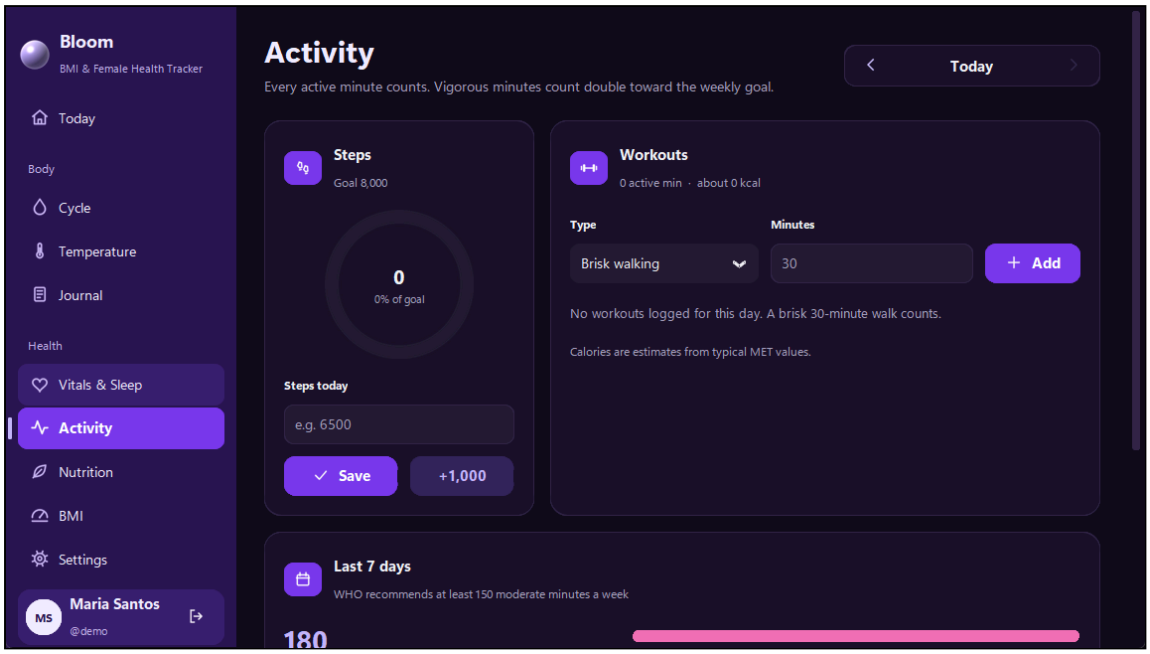
7.7 Vitals & Sleep

Log sleep hours and quality, blood pressure, resting heart rate and blood oxygen. Each reading gets a category and a colour; 14-day charts show trends.



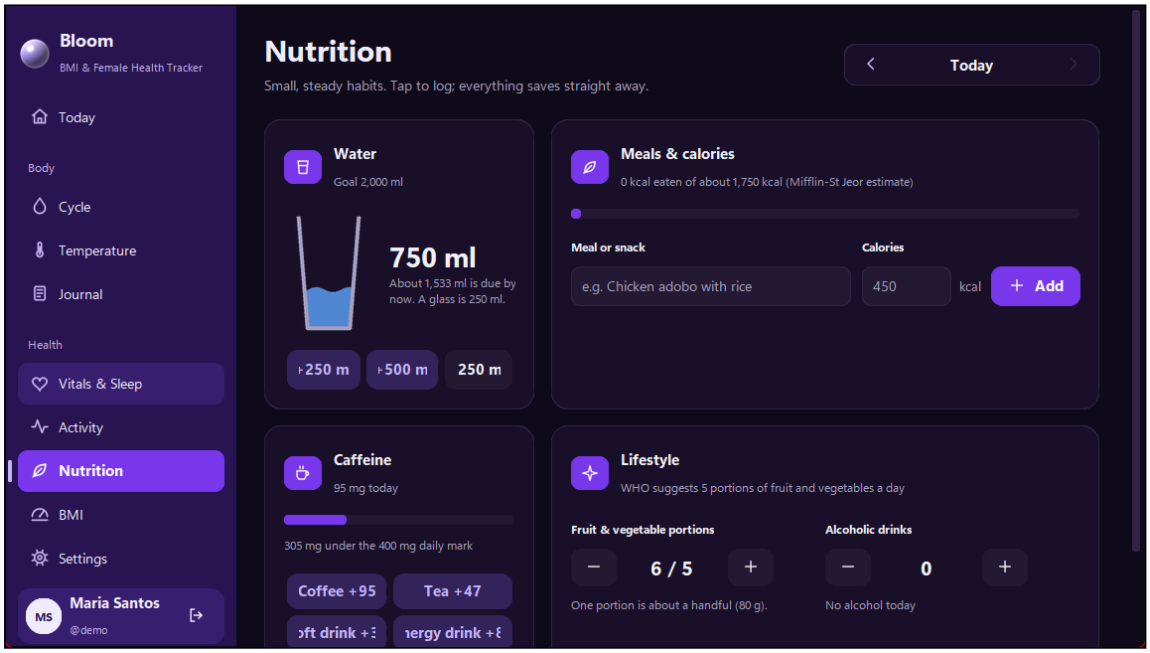
7.8 Activity

Log steps and workouts. Calories are estimated from MET values, and a ring tracks the WHO goal of 150 active minutes a week (vigorous minutes count double).



7.9 Nutrition

Add glasses of water, meals with calories, caffeinated drinks, alcohol and fruit and vegetable portions. Each has a goal or limit taken from a health guideline.



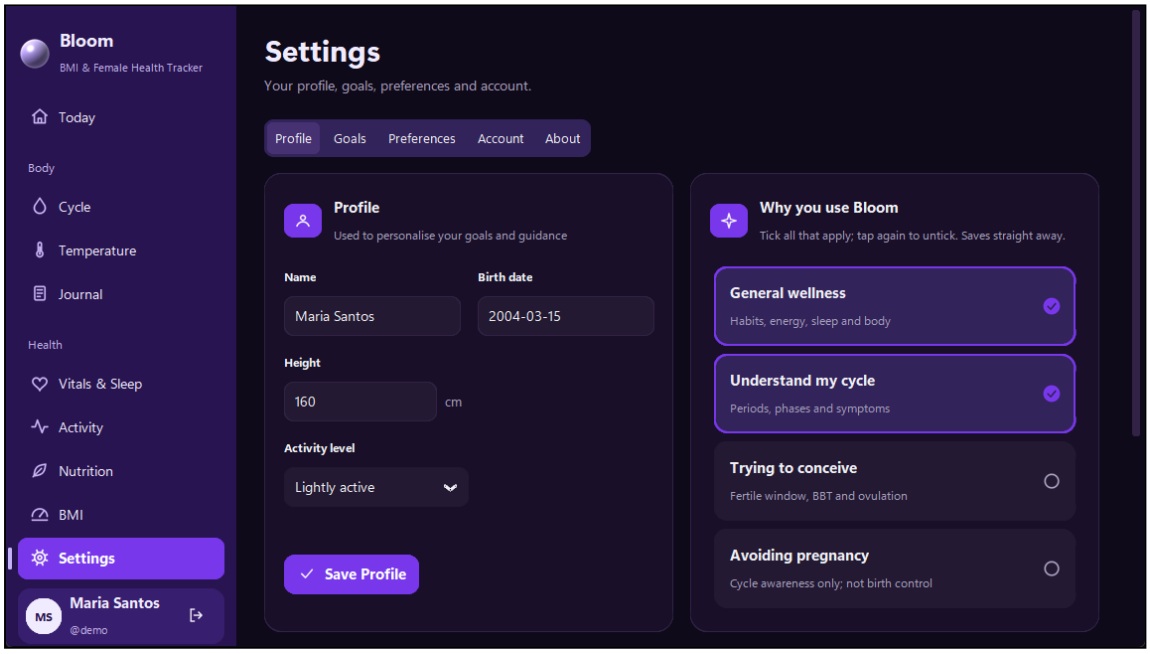
7.10 BMI

Log your weight to see your BMI on a gauge with its category (CDC or Asia-Pacific), your healthy weight range and your weight history.



7.11 Settings

Change your profile, goals, units, theme and motion; export your data as CSV; change your password or recovery code; log out or delete your account; and see the sources.



8. HEALTH STANDARDS USED

Topic	Value used in the program	Source
BMI formula	weight (kg) ÷ height (m)², rounded to 1 decimal	CDC
BMI categories (CDC)	<18.5 underweight; 18.5–24.9 healthy; 25–29.9 overweight; ≥30 obesity; adults aged 20+	CDC
BMI categories (Asia-Pacific)	<18.5; 18.5–22.9 healthy; 23–24.9 overweight; 25–29.9 obesity I; ≥30 obesity II	WHO Western Pacific Region (2000)
Normal cycle length	24–38 days	FIGO (2018)
Luteal phase	about 14 days (ovulation = next period − 14)	NHS; Office on Women's Health
Fertile window	calendar method: shortest − 18 to longest − 11	Office on Women's Health
Thermal shift	3 readings above the highest of the previous 6; 3rd ≥ 0.2 °C higher	Symptothermal method
Fever	38.0 °C (100.4 °F) or more	CDC
Blood pressure	Normal <120/<80; Elevated 120–129/<80; Stage 1 130–139 or 80–89; Stage 2 ≥140 or ≥90; Severe >180 or >120	ACC/AHA (2017), CDC
Resting heart rate	60–100 bpm	American Heart Association
Blood oxygen	95–100% typical; below 92% contact a professional	General clinical guidance
Sleep	7 or more hours a night (adults 18–60)	CDC
Physical activity	150 moderate minutes a week; vigorous counts double	WHO (2020)
Workout calories	MET × 3.5 × kg ÷ 200 per minute	Compendium of Physical Activities
Daily calories	Women: 10 × kg + 6.25 × cm − 5 × age − 161, × activity factor 1.2–1.9	Mifflin-St Jeor (1990)
Caffeine	up to 400 mg a day	FDA
Alcohol	1 drink a day or less for women	Dietary Guidelines for Americans
Fruit and vegetables	400 g (5 portions) a day	WHO
Water goal	33 ml per kg, rounded to a 250 ml glass (a design choice the user can change)	—
Password storage	PBKDF2-HMAC-SHA256, 600,000 iterations, 16-byte salt	OWASP

9. REFERENCES

1. Centers for Disease Control and Prevention. *Adult BMI Categories.* https://www.cdc.gov/bmi/adult-calculator/bmi-categories.html
2. WHO Western Pacific Region, IASO and IOTF (2000). *The Asia-Pacific Perspective: Redefining Obesity and Its Treatment.*
3. Munro, M. G., et al. (2018). The two FIGO systems for normal and abnormal uterine bleeding symptoms. *International Journal of Gynecology & Obstetrics, 143*(3), 393–408.
4. Office on Women's Health. *Your Menstrual Cycle.* https://www.womenshealth.gov/menstrual-cycle/your-menstrual-cycle
5. Office on Women's Health. *Trying to Conceive.* https://www.womenshealth.gov/pregnancy/you-get-pregnant/trying-conceive
6. NHS. *Fertility in the Menstrual Cycle.* https://www.nhs.uk/conditions/periods/fertility-in-the-menstrual-cycle/
7. Whelton, P. K., et al. (2018). 2017 ACC/AHA Guideline for the Prevention, Detection, Evaluation, and Management of High Blood Pressure in Adults. *Hypertension, 71*(6).
8. Centers for Disease Control and Prevention. *About Sleep: How Much Sleep Do I Need?* https://www.cdc.gov/sleep/about/index.html
9. World Health Organization (2020). *WHO Guidelines on Physical Activity and Sedentary Behaviour.*
10. Ainsworth, B. E., et al. *Compendium of Physical Activities.*
11. Mifflin, M. D., St Jeor, S. T., et al. (1990). A new predictive equation for resting energy expenditure in healthy individuals. *American Journal of Clinical Nutrition, 51*(2), 241–247.
12. U.S. Food and Drug Administration. *Spilling the Beans: How Much Caffeine Is Too Much?*
13. U.S. Department of Agriculture and HHS. *Dietary Guidelines for Americans, 2020–2025.*
14. World Health Organization. *Healthy Diet* fact sheet.
15. OWASP. *Password Storage Cheat Sheet.* https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
16. Python Software Foundation. *Python 3 Documentation* (datetime, calendar, hashlib, secrets, json). https://docs.python.org/3/
17. Schimansky, T. *CustomTkinter Documentation.* https://customtkinter.tomschimansky.com